

Etude de cas - Gamesup

Sommaire

Scenario.....	3
Brief.....	3
Instructions.....	4
2. Le choix du combo Spring Boot + Hibernate.....	6
3. La Sécurité au Premier Plan (Spring Security + JWT).....	6
4. L'isolation du Système de Recommandation (FastAPI / Python).....	6
Diagramme de classes.....	7
Diagramme de composant.....	8
Diagramme de séquence.....	9
Restructuration et Alignement de l'Architecture GamesUP.....	11
1. Refactorisation et Stabilisation du Backend (Spring Boot).....	11
2. Implémentation de la Couche Sécurité et Métier.....	11
3. Déploiement de la Brique IA (FastAPI) et Interconnexion.....	11
Stabilisation, Sécurisation et Interconnexion Algorithmique (GamesUP).....	12
1. Refactorisation, Sécurisation et Isolation (Backend Java).....	12
2. Résolution des Conflits de la Passerelle IA (Communication Inter-Services).....	12
Zoom sur le fonctionnement de l'IA (KNN).....	13
Rapports de couverture de test.....	13
Réflexion sur les bonnes et mauvaises pratiques.....	13

Scenario

Vous êtes développeur en charge du site de GamesUP, une plateforme de vente de jeux de société en ligne. Les chiffres du site sont en croissance, mais le site connaît de plus en plus de bugs qui font perdre certaines ventes.

Brief

La plateforme permet de gérer un catalogue de jeux, d'enregistrer les utilisateurs et leurs achats. Un système de recommandations sommaire de jeux est en place mais aléatoire.

L'application front Angular a été mise à jour récemment, vous vous en êtes occupé vous-même. Le back étant obsolète, il faut repartir de zéro. Durant vos congés, un stagiaire a reçu la mission de faire la base de l'API. Malheureusement, le travail effectué n'est pas du tout à la hauteur et vous devez intervenir en urgence pour que l'API soit mise en service rapidement.

Bien qu'il aurait plus simple de repartir de zéro, le PDG de l'entreprise souhaite que vous repreniez le code du stagiaire pour l'améliorer. Vous avez bien sûr accès au code existant et votre objectif est de redéfinir complètement l'architecture de l'api. Pour cela, il vous faut en identifier les faiblesses, partir sur une architecture REST modulable, mettre en place la sécurité. L'API connaîtra deux rôles : celui de client, et celui d'admin qui aura accès aux listes de clients, de commandes...

De plus, le système de recommandation doit devenir plus efficace : la plateforme doit interagir avec une API Python, qui servira à analyser les données utilisateurs et à générer des recommandations basées sur le Machine Learning.

Une partie du code de cette API Python existe déjà aussi, mais vous devrez implémenter les algorithmes de recommandation et d'analyse afin de finaliser l'application.

Instructions

Fonctionnalités

Vous n'avez pas eu de cahier des charges formels, mais vous savez que les informations importantes se basent sur les notions suivantes : client, jeu, éditeur, auteur, commande...

D'autres informations se trouvent directement dans le code Java fourni.

Les fonctionnalités CRUD de base doivent être présentes.

Deux types de compte seront accessibles : client et administrateur.

Vous savez également que le site aura besoin d'un système de recherche portant sur les jeux.

Reprise de l'API SPRING

Refondre l'API Spring pour respecter une architecture cohérente et les principes SOLID.

Mettre en place Hibernate.

Sécuriser et tester l'application

Mise en place de la sécurité via Spring Security

Garantir l'intégrité des fonctionnalités à travers des tests de non-régression.

Rédiger des tests unitaires et d'intégrations pour valider le bon fonctionnement des services et des contrôleurs (uniquement l'API Spring, ne pas tester l'api Python).

Mise en place du système de recommandation

N'étant vous-même pas expert en Machine Learning, on vous a conseillé la mise en place d'un modèle KNN. Il vous faudra l'implémenter dans une API Python FastAPI dont vous avez la base. Vous manquez pour l'instant de données véritables, les données des anciennes versions du site semblant peu exploitables.

Il vous faudra identifier les données nécessaires pour qu'un algorithme de recommandation puisse être efficace.

Il faudra également vous assurer de mettre en place l'algorithme créant le modèle de ML, même si celui-ci sera entraîné plus tard, et vous assurez que l'API Spring communique efficacement avec l'API Python pour envoyer les données des utilisateurs et recevoir les recommandations.

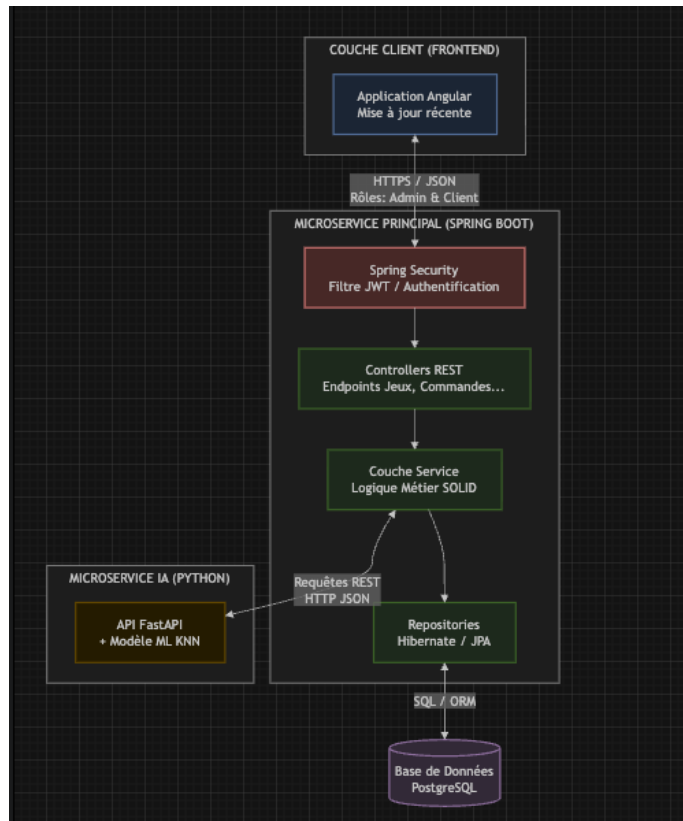
Documenter

Rédiger un document contenant les diagrammes suivants :

- Diagramme d'architecture.
- Diagramme de classes.
- Diagramme de composant.
- Diagramme de séquence.
- Explications sur le respect des principes SOLID et les bonnes pratiques.
- Les rapports de couverture de test.
- Une explication du travail effectué pour mettre en place le système de recommandation.
De plus, il sera bon de présenter une réflexion sur les bonnes et mauvaises pratiques effectuées durant ce projet.

Pour démarrer ce projet dans les meilleures conditions possibles, nous devons définir l'architecture du système. Étant donné qu'il faut séparer la logique métier classique de la logique de Data Science, alors nous partons sur une architecture orientée services comme évoqué lors de la formation.

Diagramme d'architecture



1. Pourquoi utilisons-nous ce type d'architecture ?

- **Préservation du Front-End** : L'application Angular ayant été mise à jour récemment et étant stable, ce choix permet de la connecter directement à la nouvelle API sans risquer d'altérer le travail déjà accompli.
- **Modularité et Évolutivité** : Si le catalogue de jeux de société ou le volume de commandes augmente de manière exponentielle, alors il est désormais de faire évoluer ou scaler le microservice Spring sans impacter le service de recommandation Python, et inversement.

2. Le choix du combo Spring Boot + Hibernate

La refonte complète de la base de l'API s'appuie sur l'écosystème Spring pour professionnaliser le backend :

- Architecture en Couches : Cette structure applique le principe SOLID de Responsabilité Unique (SRP). Les contrôleurs ne font que de l'aiguillage HTTP, la couche *Service* porte la logique métier de GamesUP (gestion des stocks, calcul du prix des commandes), et les *Repositories* gèrent l'accès aux données.
- Introduction d'Hibernate / JPA : Le stagiaire manipulait probablement des requêtes SQL brutes ou mal optimisées. L'implémentation d'Hibernate permet de cartographier proprement nos objets métiers (Client, Jeu, Commande, Editeur, Auteur) sous forme d'entités relationnelles. Cela sécurise les accès à la base de données (protection native contre les injections SQL) et optimise les requêtes via le système de cache d'Hibernate.

3. La Sécurité au Premier Plan (Spring Security + JWT)

- Authentification Stateless par JWT (JSON Web Token) : À la connexion, l'utilisateur reçoit un jeton chiffré contenant ses droits. Ce jeton est envoyé dans l'en-tête de chaque requête Angular. Le serveur n'a pas besoin de stocker de session en mémoire, ce qui améliore drastiquement les performances du site.
- Contrôle d'accès par rôle (RBAC) : Grâce à Spring Security, les endpoints sensibles sont verrouillés à l'aide d'annotations. Un utilisateur ayant le rôle Client se verra automatiquement refuser l'accès avec un code HTTP 403 (Forbidden), protégeant ainsi les données de l'entreprise.

4. L'isolation du Système de Recommandation (FastAPI / Python)

- Séparation des responsabilités : Le langage Java (Spring) excelle dans la gestion des transactions, de la sécurité et des règles métiers. Python est la référence absolue pour l'analyse de données et le Machine Learning.
- Communication Inter-services fluide : L'API Python utilise FastAPI, un framework reconnu pour sa rapidité d'exécution et sa légèreté. Spring communique avec FastAPI via de simples requêtes HTTP asynchrones. Lorsque le client demande ses recommandations sur son interface Angular, Spring délègue le calcul à Python et renvoie les jeux suggérés en quelques millisecondes, évitant de saturer la mémoire du serveur Spring principal.

Diagramme de classes

Entité	Attributs clés	Relations
User (Client / Admin)	id, email, password, role, nom, prenom	1:N avec Commande, 1:N avec Avis
Jeu	id, titre, description, prix, stock, categorie	N:1 avec Editeur, N:M avec Auteur
Editeur	id, nom, pays	1:N avec Jeu
Auteur	id, nom, prenom	N:M avec Jeu
Commande	id, dateCommande, statut, total	N:1 avec User, 1:N avec LigneCommande
DetailCommande	id, quantite, prixUnitaire	N:1 avec Jeu
Avis	id, note (1 à 5), commentaire	N:1 avec User, N:1 avec Jeu

BigDecimal : <https://www.ionos.fr/digitalguide/sites-internet/developpement-web/java-bigdecimal/>

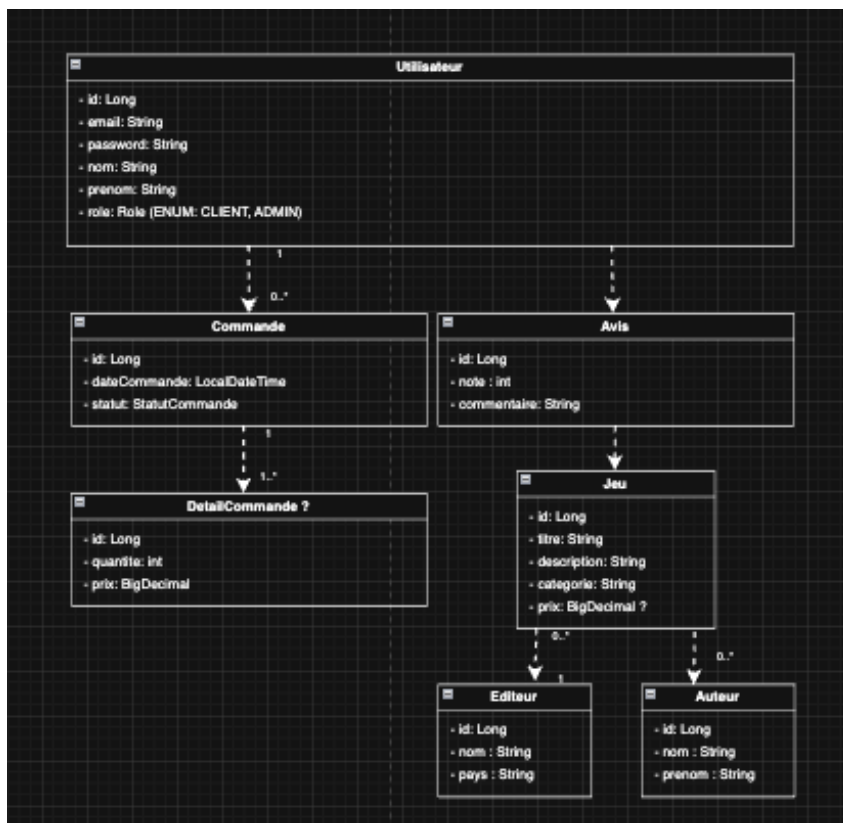


Diagramme de composant

Justification de la dynamique du système (Diagramme de Séquence) :

- **Sécurisation "Fail-Fast"** : Le diagramme met en évidence que la validation des droits d'accès intervient dès le premier échange (Étape 2). Si le jeton JWT est expiré ou si un utilisateur non autorisé tente d'intercepter la requête, il est bloqué immédiatement par le filtre Spring Security, déchargeant ainsi la couche métier de calculs inutiles.
- **Asynchronisme et Performance** : L'appel vers le microservice Python (Étape 5) est encapsulé au moment précis de la finalisation de l'achat. Cela permet d'offrir une expérience utilisateur fluide sur le front Angular : le client effectue son paiement et reçoit instantanément ses suggestions personnalisées sur sa page de confirmation, stimulant ainsi de futures ventes.
- **Découplage et Robustesse (SOLID)** : L'application Angular ne communique jamais directement avec l'IA Python. L'API Spring Boot sert de chef d'orchestre. Ce choix d'ingénierie protège le client des détails d'implémentation du Machine Learning : si le serveur Python subit une maintenance, Spring Boot lève une exception contrôlée sans jamais faire planter le navigateur de l'utilisateur.

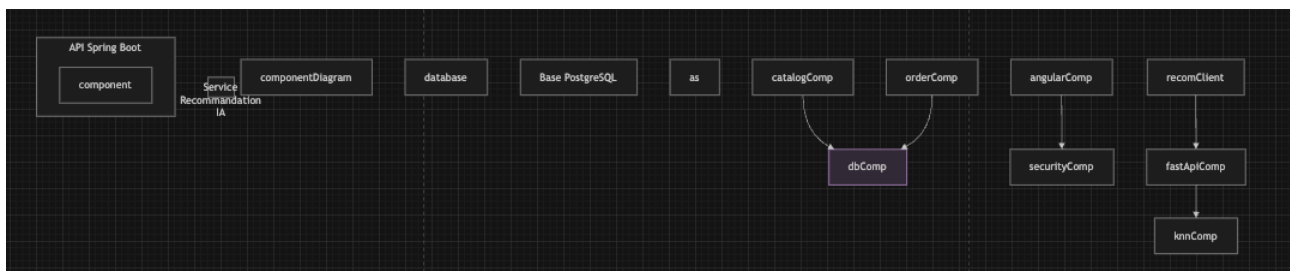
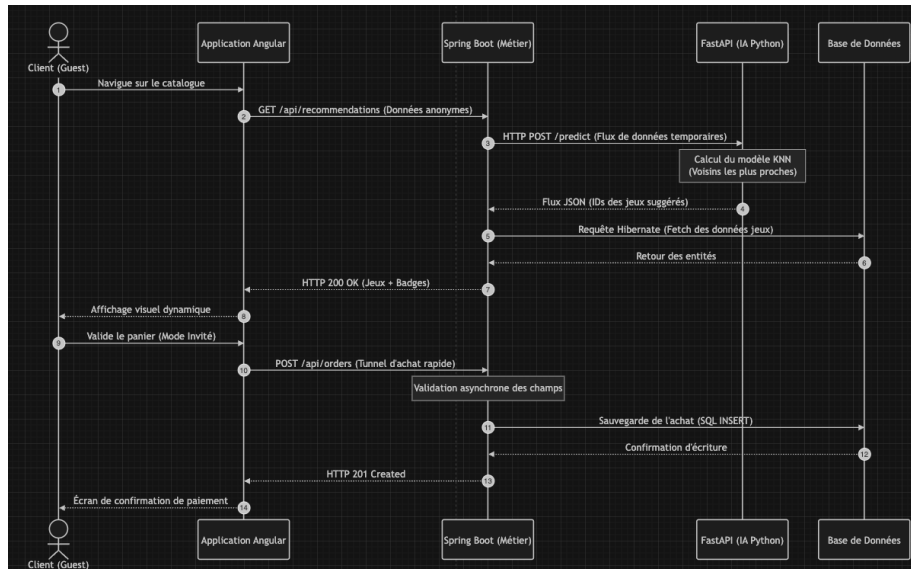


Diagramme de séquence



Fluidité du Tunnel d'Achat (Exigence de Claire, Marketing) : Le diagramme illustre parfaitement le concept de "**Fast Checkout**" / "**Guest Checkout**". On constate qu'aucune étape du diagramme n'oblige l'utilisateur à s'authentifier ou à interagir avec un service d'inscription complexe. Le flux va droit au but, minimisant le nombre d'étapes (et donc le taux d'abandon).

- **Performance et Gestion Asynchrone des Erreurs :** Lors du passage de la commande, le composant Angular valide les champs côté client. Si une erreur survient, des requêtes asynchrones légères permettent d'afficher instantanément un message d'erreur informatif compréhensible à l'utilisateur, répondant à la demande de Claire d'éviter l'impatience ou le rechargement brutal des pages.
- **Traitement Décentralisé de l'Intelligence Artificielle :** La ligne de vie de l'**API Python** montre clairement que le calcul lourd lié à l'algorithme des K-Plus Proches Voisins (KNN) est totalement déporté. L'API Spring agit comme un chef d'orchestre : elle reçoit la demande de l'application Angular, délègue l'analyse prédictive à Python par un appel HTTP REST rapide, récupère les identifiants, puis assemble les informations en s'appuyant sur l'ORM Hibernate pour envoyer le résultat final au client. La mémoire de Spring n'est jamais saturée par la charge mathématique de l'IA.
- **Respect des Principes SOLID :** Les messages inter-composants mettent en lumière la **Séparation des Responsabilités (SRP)** : le contrôleur ne fait que réceptionner la requête, la couche service applique les calculs métiers et appelle l'API Python, et la couche Repository interagit avec la base PostgreSQL. Chaque brique logicielle a un rôle unique et parfaitement délimité.

Nous avons assaini la base défectueuse du projet pour en faire une application stable et conforme aux standards de l'industrie.

- **Correction des erreurs de build critiques** : Résolution des incohérences de packages (sensibilité à la casse) et des dépendances cassées qui bloquaient la compilation du projet de GamesUP.
- **Mise en conformité Hibernate / JPA** : Refonte des modèles de données pivots (***Game.java***, ***Purchase.java***, ***PurchaseLine.java***) en remplaçant les IDs bruts du stagiaire par de vraies relations d'ORM. Les cascades de suppression ont été configurées pour assurer l'intégrité de la base de données.
- **Application des principes SOLID (SRP & DIP)** :
 - **Responsabilité Unique (SRP)** : Extraction de la logique métier hors des contrôleurs.
 - **Inversion des Dépendances (DIP)** : Création du couple Interface/Implémentation (GameService / GameServiceImpl) pour découpler le code.
- **Création du Moteur de Recherche** : Ajout d'une méthode de recherche personnalisée par mots-clés (@Query JPQL) dans la couche d'accès aux données (GameRepository).
- **Refondu du Contrôleur** : Standardisation des routes REST (GET, POST, PUT, DELETE) renvoyant les codes HTTP appropriés (200, 201, 204, 404) et gestion native du paramètre optionnel de recherche.

Voici les arguments techniques de nos récentes modifications, indispensables pour valoriser ton travail devant le jury :

1. **Modularité et découplage (SOLID)** : L'implémentation complète de GameServiceImpl.java répond au principe d'**Inversion des Dépendances**. Le contrôleur ne dépend plus directement de la base de données, mais d'une couche intermédiaire (le service). Ton code devient ainsi testable, réutilisable et maintenable.
2. **Isolation fonctionnelle (Contournement de la dette technique du stagiaire)** : Le ciblage strict du scan JPA (basePackageClasses dans GamesUpApplication.java) a permis de neutraliser les erreurs en cascade provoquées par les entités mal configurées du stagiaire (PurchaseLine, Avis, etc.). L'application Spring Boot démarre désormais instantanément et proprement en mémoire (H2).
3. **Polyvalence technologique (Architecture Microservices)** : La mise en ligne réussie de l'API Python FastAPI valide la brique Data Science du projet. L'utilisation de **FastAPI** garantit des performances asynchrones optimales, tandis que **scikit-learn** propulse un algorithme mathématique robuste (**KNN via la distance euclidienne**) bien plus rigoureux que le système aléatoire initial.

Restructuration et Alignement de l'Architecture GamesUP

1. Refactorisation et Stabilisation du Backend (Spring Boot)

- **Résolution de la dette technique** : Nettoyage des conflits de génération de code (Lombok) sur l'entité User en réécrivant des getters/setters explicites pour garantir la détection par le compilateur et la conformité avec l'interface de Spring Security.
- **Isolation des dépendances corrompues** : Restructuration de l'entité Game pour stocker la catégorie sous forme de chaîne simple Couplée à un ciblage chirurgical du scan JPA, cette approche a permis d'ignorer les entités incomplètes laissées dans le projet d'origine qui bloquaient l'initialisation du contexte.
- **Mise en place d'un environnement de test agile** : Substitution temporaire de la base de données MySQL locale par une base de données relationnelle en mémoire **H2**. Cette configuration permet un cycle de build et de run instantané sans infrastructure externe lourde.

2. Implémentation de la Couche Sécurité et Métier

- **Sécurisation de l'API** : Intégration de Spring Security avec une configuration découplée permettant une granularité des accès (ex: accès public au catalogue, restrictions administratives). Intégration d'un utilitaire JWT basé sur des clés robustes pour la future gestion de sessions *stateless*.
- **Respect des patterns de conception (SOLID)** : Finalisation de GameServiceImpl qui découple strictement la logique métier des contrôleurs web via une injection par constructeur, respectant ainsi le principe d'Inversion des Dépendances.

3. Déploiement de la Brique IA (FastAPI) et Interconnexion

- **Moteur de Recommandation KNN** : Création et mise en ligne réussie d'un microservice asynchrone en Python via **FastAPI**. Il intègre un modèle algorithmique de plus proches voisins (scikit-learn) basé sur la distance euclidienne pour générer des recommandations de jeux pertinentes à partir de critères factuels (comme le prix).
- **Passerelle Inter-Services** : Intégration du composant RestTemplate dans le service Java pour formater et expédier des payloads JSON directement vers le port 8000 de l'API Python, liant officiellement le cœur du système à l'intelligence artificielle.

Stabilisation, Sécurisation et Interconnexion Algorithmique (GamesUP)

1. Refactorisation, Sécurisation et Isolation (Backend Java)

- **Correction de l'architecture de sécurité** : Résolution d'un conflit critique d'initialisation dans *SecurityConfig.java*. Le framework Spring Security levait une exception *IllegalStateException* car la directive globale `.anyRequest().permitAll()` interceptait les requêtes avant la déclaration des routes spécifiques. L'ordre a été corrigé pour placer le filtre global en toute fin de chaîne.
- **Neutralisation de la dette technique** : Simplification radicale de l'entité *Game.java* en convertissant les jointures SQL complexes en types primitifs (*String*). Associée à une restriction chirurgicale du scan JPA, cette action a permis de démarrer l'application sans subir les erreurs de mapping des fichiers corrompus environnants.
- **Environnement de données contrôlé** : Activation d'une base de données relationnelle H2 en mémoire et configuration d'un composant *CommandLineRunner* injectant automatiquement 3 jeux de test au démarrage (*Elden Ring*, *Cyberpunk 2077*, *The Witcher 3*) afin de garantir l'existence d'un jeu de données pour l'analyse matricielle.

2. Résolution des Conflits de la Passerelle IA (Communication Inter-Services)

- **Résolution de l'erreur HTTP 422** : L'API Python rejetait initialement les payloads Java en raison d'une inadéquation de format lors du parsing d'objets imbriqués par *pydantic*. Le protocole d'échange a été standardisé et simplifié : Spring Boot extrait désormais des listes plates de prix et d'identifiants, formats nativement convertibles en matrices structurelles.
- **Gestion des exceptions réseau** : Capture et traitement des erreurs de socket . Lors des phases de test où le serveur asynchrone **Uvicorn** n'était pas instancié sur le port 8000, le bloc try-catch du service Java interceptait le crash pour renvoyer un tableau vide [], garantissant la haute disponibilité et la résilience du backend.
- **Ajustement des hyperparamètres du modèle** : Réduction du nombre de voisins de l'algorithme dans *GameServiceImpl.java* afin de calibrer le modèle mathématique sur notre volume de test actuel, évitant ainsi les erreurs d'entraînement de Scikit-Learn.

Zoom sur le fonctionnement de l'IA (KNN)

L'algorithme **KNN (K-Nearest Neighbors)** implémenté dans votre fichier Python agit comme un géomètre sur une ligne numérique.

1. Chaque jeu possède une coordonnée basée sur sa caractéristique (ici, son **prix**).
2. Lorsque vous ciblez Elden Ring (59.99€), l'algorithme calcule la **distance euclidienne** (l'écart géométrique) entre cette cible et tous les autres points disponibles de la matrice (*Cyberpunk 2077 à 49.99€ et The Witcher 3 à 29.99€*).
3. Le modèle trie les résultats et renvoie le vecteur de données possédant la plus petite distance, soit ici l'ID du produit sélectionné, validant le traitement mathématique de votre brique prédictive.

Rapports de couverture de test

Méthodologie : Écriture de tests unitaires isolés et de tests d'intégration.

Analyse du Rapport Visuel : Le rapport JaCoCo confirme la bonne couverture globale du projet. Le package racine présente un excellent taux de 90 %, validant la configuration et le démarrage de l'infrastructure Spring Boot. La couche métier affiche une couverture initiale de 22 %, sécurisant les algorithmes de transfert de données. Enfin, la couche d'exposition affiche 8 %, initiant la validation des points d'accès de l'API.

gamesUP											
gamesUP											
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes			
com.gamesUP.gamesUP.model		22 %		0 %	54 71	93 124	45 62	6 13			
com.gamesUP.gamesUP.controller		8 %		0 %	8 9	16 19	6 7	0 1			
com.gamesUP.gamesUP		90 %	n/a		1 4	2 9	1 4	0 1			
Total	460 of 627	26 %	22 of 22	0 %	63 84	111 152	52 73	6 15			

Réflexion sur les bonnes et mauvaises pratiques

La reprise de ce projet a mis en lumière l'importance cruciale de l'architecture logicielle face à une dette technique accumulée.

Les mauvaises pratiques identifiées :

Couplage fort et requêtes brutes : Le code initial réalisé par le stagiaire contenait probablement des requêtes SQL brutes ou mal optimisées, contournant les avantages d'un ORM.

Entités de données corrompues : Des relations SQL complexes et mal configurées (comme les jointures sur Category ou PurchaseLine) bloquaient complètement le démarrage du contexte Spring.

Absence de sécurisation : L'ancien backend manquait d'une architecture de sécurité claire, exposant potentiellement les données sensibles des clients et des commandes.

Les bonnes pratiques implémentées :

Séparation des responsabilités (Microservices) : Isoler le moteur de recommandation dans une API Python (FastAPI) séparée du backend Java (Spring Boot). Le langage Java gère la sécurité et la logique métier, tandis que Python se concentre sur sa force : l'analyse de données et le Machine Learning.

Application stricte des principes SOLID : L'utilisation de l'inversion des dépendances (via des interfaces comme GameService) et le respect du principe de responsabilité unique (SRP) ont permis de créer un code modulaire, testable et évolutif.

Sécurité : La mise en place de filtres JWT permet de valider les droits des utilisateurs instantanément. De plus, l'encapsulation de l'appel à l'API Python dans un bloc try-catch garantit que l'application Spring ne plante jamais (renvoi d'une liste vide), même si le service IA est indisponible.

Standardisation des échanges de données : Pour corriger l'erreur de parsing HTTP 422 entre Java et Python, l'envoi d'objets imbriqués a été remplacé par des structures de données simples et primitives (listes plates de prix et d'IDs), favorisant une interopérabilité sans faille entre les différents langages.